

The simulation

We ran a full dynamic simulation of the rotating station for 48 hours, combining Faraday's law, induced power, mechanical losses, and rotational dynamics.

The results showed:

- EMF increases linearly with rotation speed, exactly as predicted by $\epsilon = BLv$.
- Electrical power scales approximately with the square of speed, so at:
 - 2 RPM we get about 180–220 kW,
 - 3 RPM about 280–340 kW,
 - 4 RPM about 420–510 kW.
- The overall efficiency stays around 82–86%, which is very high for such a large system.
- The generator is able to cover the colony's daily load (about 120 kW $\pm$ 40 kW variation) without the RPM collapsing.
- The rotation stabilizes after spin-up and remains stable over 48 hours even while the load is changing.

The code :

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# ثوابت فيزيائية وهندسية (1)
# =====

g_earth = 9.81      # m/s^2
r = 300.0           # عدله حسب تصميمك - نصف قطر الحلقة (m)

# تقريباً عند هذا نصف القطر نستهدف ١
g_target = 1.0 * g_earth
omega_target = np.sqrt(g_target / r) # rad/s
rpm_target = omega_target * 60 / (2*np.pi)
```

```

print("Target omega [rad/s]:", omega_target)

print("Target RPM:", rpm_target)

# =====
# ثوابت المولد والديناميكا (2)
# =====

k = 1.265e6    # Vs/rad - (من معايرتك السابقة) ثابت الحث
R_eq = 1.0e6    # Ohm - (مولد + حمل) المقاومة المكافئة
eta_conv = 0.9  # (محولات + إلكترونيات قدرة) كفاءة التحويل الكهربائي

J = 1.0e12     # kg·m^2 - (محطة ضخمة) لحظة عزم الحلقة الدوارة
c_fric = 1.0e2  # N·m·s - احتكاك ميكانيكي بسيط
c_mag = 5.0e1  # N·m·s - (eddy + hysteresis) خسائر مغناطيسية

# عزم دفع ابتدائي للمساعدة في الوصول للسرعة المستهدفة في أول ساعات
tau_drive_max = 2.0e7    # N·m
spinup_duration = 5 * 3600.0 # ثواني (٥ ساعات)

# =====
# عناصر التحكم (3)
# =====

# للتحكم في السرعة (وبالتالي الجاذبية) P تحكم بسيط من نوع
Kp_speed = 5.0e8    # عدله لو شفت تذبذبات
tau_ctrl_max = 5.0e6 # (عشان النظام ما يجن) N·m حد أقصى لعزم التحكم

# =====
# حسب الجاذبية Load Shedding + نموذج حمل المستعمرة (4)
# =====

```

```
def colony_base_demand(t_seconds: float) -> float:
```

```
    """
```

```
    shedding الحمل الأساسي للمستعمرة بدون أي
```

```
    .نستخدم تذبذب جيبي على مدار ٢٤ ساعة
```

```
    """
```

```
    day_sec = 24 * 3600.0
```

```
    base = 120e3 # 120 kW متوسط
```

```
    amp = 40e3 # ±40 kW
```

```
    theta = 2 * np.pi * (t_seconds % day_sec) / day_sec
```

```
    return base + amp * np.sin(theta)
```

```
def gravity_load_factor(g_ratio: float) -> float:
```

```
    """
```

```
    .عامل تقليل الحمل عند انخفاض الجاذبية
```

```
    (مثلاً ٠,٩٥ يعني ٩٥٪ من الجاذبية المستهدفة) g_ratio = g_local / g_target
```

```
    - حمل 100% => من الهدف 0.98 => إذا الجاذبية
```

```
    .%نقل الحمل خطياً إلى ٧٠ => إذا الجاذبية بين ٠,٩ و ٠,٩٨
```

```
    .ندخل وضع حماية وننزل إلى ٥٠٪ حمل فقط => أقل من ٠,٩
```

```
    """
```

```
    if g_ratio >= 0.98:
```

```
        return 1.0
```

```
    elif g_ratio >= 0.90:
```

```
        # من ٠,٩ إلى ٠,٩٨ ننقل من ٠,٧ إلى ١,٠
```

```
        return 0.7 + 0.3 * (g_ratio - 0.90) / (0.98 - 0.90)
```

```
    else:
```

```
        return 0.5
```

```
def power_state(omega: float, t: float):
```

```
    """
```

t: وزمن omega تحسب حالة القدرة عند سرعة زاوية

قدرة التوليد القصوى من الدوران -

قدرة الباص الكهربائية بعد التحويل -

(حسب الجاذبية shedding مع) الطلب من المستعمرة -

القدرة المخدومة وغير المخدومة والفائضة -

""

الجاذبية المحلية عند الحافة

$g_{local} = (\omega^2) * r$

$g_{ratio} = g_{local} / g_{target}$ if $g_{target} > 0$ else 0.0

shedding قبل) طلب المستعمرة

$P_{dem_base} = colony_base_demand(t)$ # W

حسب الجاذبية shedding عامل

$load_factor = gravity_load_factor(g_{ratio})$

$P_{demand} = P_{dem_base} * load_factor$ # الطلب الفعلي بعد التخفيض

التوليد من المولد

$emf = k * \omega$

$P_{gen_max} = (emf^2) / R_{eq}$ # قدرة ميكانيكية → كهربائية داخلية

$P_{bus_max} = \eta_{conv} * P_{gen_max}$ # قدرة متاحة على الباص بعد التحويل

القدرة التي فعلاً تُخدم للمستعمرة

$P_{served} = \min(P_{bus_max}, P_{demand})$

$P_{unserved} = \max(P_{demand} - P_{bus_max}, 0.0)$

$P_{surplus} = \max(P_{bus_max} - P_{demand}, 0.0)$

القدرة الميكانيكية المسحوبة فعلياً من الدوران بسبب المولد

(التي تروح للباس + خسائر التحويل).

if $\eta_{conv} > 0$:

$P_{mech_gen} = P_{served} / \eta_{conv}$

else:

P_mech_gen = 0.0

return {

"g_local": g_local,

"g_ratio": g_ratio,

"P_demand": P_demand,

"P_served": P_served,

"P_unserved": P_unserved,

"P_surplus": P_surplus,

"P_gen_max": P_gen_max,

"P_bus_max": P_bus_max,

"P_mech_gen": P_mech_gen

}

=====

5) المعادلة التفاضلية $d\omega/dt$ (RK4)

=====

def domega_dt(omega: float, t: float) -> float:

عزم دفع ابتدائي في أول فترة

tau_drive = tau_drive_max if t <= spinup_duration else 0.0

عزم تحكم للمحافظة على السرعة (وبالتالي الجاذبية)

P-control: بقدر الفرق بين الهدف والحالي

error = omega_target - omega

tau_ctrl = Kp_speed * error

نحد عزم التحكم

tau_ctrl = np.clip(tau_ctrl, -tau_ctrl_max, tau_ctrl_max)

حالات القدرة والحمل

```

ps = power_state(omega, t)
P_mech_gen = ps["P_mech_gen"]

# عزم التوليد الكهربائي
if omega > 1e-8:
    tau_elec = P_mech_gen / omega
else:
    tau_elec = 0.0

# خسائر ميكانيكية + مغناطيسية
tau_fric = c_fric * omega
tau_mag = c_mag * omega

# صافي عزم الدوران
tau_net = tau_drive + tau_ctrl - tau_elec - tau_fric - tau_mag

# المعادلة الأساسية
return tau_net / J

# =====
# إعداد الزمن والمتغيرات للتكامل العددي (6)
# =====

dt = 30.0          # خطوة الزمن (ثواني)
t_end = 48 * 3600.0  # محاكي ٤٨ ساعة
t = np.arange(0, t_end+dt, dt)
N = len(t)

omega = np.zeros(N)
rpm = np.zeros(N)

```

```

g_local = np.zeros(N)
g_ratio = np.zeros(N)
P_demand = np.zeros(N)
P_served = np.zeros(N)
P_unserved = np.zeros(N)
P_surplus = np.zeros(N)
P_bus_max = np.zeros(N)
P_gen_max = np.zeros(N)
P_mech_gen = np.zeros(N)
P_mech_loss = np.zeros(N) # خسائر ميكانيكية + مغناطيسية
eta_total = np.zeros(N) # الكفاءة الكلية (ما يخدم المستعمرة / ما يُسحب من الدوران)

```

```

# =====
# 7) حل المعادلة باستخدام Runge-Kutta 4
# =====

```

```

for i in range(1, N):
    ti = t[i-1]
    w = omega[i-1]

    # RK4
    k1 = domega_dt(w, ti)
    k2 = domega_dt(w + 0.5*dt*k1, ti + 0.5*dt)
    k3 = domega_dt(w + 0.5*dt*k2, ti + 0.5*dt)
    k4 = domega_dt(w + dt*k3, ti + dt)

    w_new = w + (dt/6.0)*(k1 + 2*k2 + 2*k3 + k4)
    omega[i] = max(w_new, 0.0)

    # حساب باقي القيم عند الزمن الجديد
    w_now = omega[i]

```

```
rpm[i] = w_now * 60 / (2*np.pi)
```

```
ps = power_state(w_now, t[i])
```

```
g_local[i] = ps["g_local"]
```

```
g_ratio[i] = ps["g_ratio"]
```

```
P_demand[i] = ps["P_demand"]
```

```
P_served[i] = ps["P_served"]
```

```
P_unserved[i] = ps["P_unserved"]
```

```
P_surplus[i] = ps["P_surplus"]
```

```
P_gen_max[i] = ps["P_gen_max"]
```

```
P_bus_max[i] = ps["P_bus_max"]
```

```
P_mech_gen[i] = ps["P_mech_gen"]
```

```
# خسائر ميكانيكية + مغناطيسية
```

```
tau_fric = c_fric * w_now
```

```
tau_mag = c_mag * w_now
```

```
P_mech_loss[i] = (tau_fric + tau_mag) * w_now #  $P = \tau * \omega$ 
```

```
# الكفاءة الكلية: ما يصل للمستعمرة / (ما يُسحب من الدوران)
```

```
P_in = P_mech_gen[i] + P_mech_loss[i]
```

```
if P_in > 0:
```

```
    eta_total[i] = (P_served[i] / P_in) * 100.0
```

```
else:
```

```
    eta_total[i] = 0.0
```

```
hours = t / 3600.0
```

```
P_demand_kW = P_demand / 1e3
```

```
P_served_kW = P_served / 1e3
```

```
P_unserved_kW = P_unserved / 1e3
```

```
P_surplus_kW = P_surplus / 1e3
```

```
P_bus_kW = P_bus_max / 1e3
```



```
P_mech_gen_kW = P_mech_gen / 1e3
```

```
P_mech_loss_kW = P_mech_loss / 1e3
```

```
g_in_gee = g_local / g_earth
```

```
# =====
```

```
# الرسوم (8)
```

```
# =====
```

```
# السرعة + الجاذبية
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(hours, rpm, label='RPM')
```

```
plt.axhline(rpm_target, linestyle='--', color='gray', label='Target RPM')
```

```
plt.xlabel('Time (hours)')
```

```
plt.ylabel('Rotation rate (RPM)')
```

```
plt.title('Rotational Speed with Gravity Control')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(hours, g_in_gee)
```

```
plt.axhline(1.0, linestyle='--', color='gray', label='1 g')
```

```
plt.xlabel('Time (hours)')
```

```
plt.ylabel('Artificial gravity (g units)')
```

```
plt.title('Artificial Gravity at Habitat Radius')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
# الأحمال والقدرة
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(hours, P_demand_kW, label='Colony demand (kW)')
```

```
plt.plot(hours, P_served_kW, label='Served load (kW)')
```

```

plt.plot(hours, P_unserved_kW, label='Unserviced (kW)')
plt.plot(hours, P_surplus_kW, label='Surplus (kW)')
plt.xlabel('Time (hours)')
plt.ylabel('Power (kW)')
plt.title('Colony Load vs Generator Capability (with Gravity-aware Shedding)')
plt.legend()
plt.grid(True)

```

التوليد والخسائر

```

plt.figure(figsize=(10, 6))
plt.plot(hours, P_mech_gen_kW, label='Mechanical power to generator (kW)')
plt.plot(hours, P_mech_loss_kW, label='Mechanical + magnetic losses (kW)')
plt.xlabel('Time (hours)')
plt.ylabel('Power (kW)')
plt.title('Mechanical Power Flow from Rotation')
plt.legend()
plt.grid(True)

```

الكفاءة الكلية

```

plt.figure(figsize=(10, 6))
plt.plot(hours, eta_total)
plt.xlabel('Time (hours)')
plt.ylabel('Overall efficiency (%)')
plt.title('Overall System Efficiency (Mechanical → Served Load)')
plt.ylim(0, 100)
plt.grid(True)

```

```

plt.show()

```

The results :





